

Executive summary — Visual Augmentation of Audio Semantics for Accessibility

For someone taking over this project today (updated 2026-09-14, 19:00). Plain language, every claim backed by a file you can open. The detailed record is `docs/project_notes.tex` (the dated lab notebook), `docs/literature_review.tex` (background), `docs/report/report.pdf` (the thesis draft, 14 pages; bibliography entries marked [verify] still need checking), `LIMITATIONS.md` (what the numbers do not support) and `docs/failure_catalogue.md` (every failure seen, with its status).

1. The problem in one paragraph

A deaf viewer gets a video's speech through subtitles but not the rest of its soundtrack: a siren behind the camera, a dog barking in the next room, glass breaking off screen. Caption guidelines even tell captioners to skip any sound the picture already shows, and in practice most ambient sound is never captioned. This project built a system that hears a video's ambient sounds, decides for each whether its source is **already visible on screen**, and — only when it is not — shows a small picture of that sound beside the video while it lasts. The "only when not visible" rule is called the **gate**; it is the project's central idea and the thing the evaluation tests.

2. Background: what exists and why it is not enough

- **Subtitles/SDH** carry speech well and non-speech sound badly and inconsistently; sign language is richer but scarce and not automatable.
- **Sound-awareness tools for deaf users** (HomeSound, SoundWatch, ProtoSound) recognise ~20 everyday sounds in the real world; users prefer icons/pictures to text. They do not address media.
- **Audio event detection** is mature: models trained on AudioSet (527 sound classes) name what is in a soundtrack. We use **BEATs** (PANNs was first and was replaced).
- **Vision-language models (VLMs)** answer open questions about frames — the only way to ask "is the thing making this sound visible?" without a fixed list of objects.

- **Audio-to-image generation** (Sound2Scene etc.) turns sound into a picture without looking at the video. Those systems are our **blind baseline**: "draw everything you hear".
- We found no prior system that combines detect → reason about heard-but-not-seen → show a picture. That combination, its benchmark and its evaluation are the contribution.

Constraints from the supervisor: **no training** (glue existing open models), **minimum human involvement** (decisions by models, not hand-written rules), finish this week.

3. What the system does (seven stages, `src/`)

stage	what	model
1	extract audio	FFmpeg
2	quick check of which objects appear anywhere in the clip; stage 5 makes the actual visibility decision	OWLv2 on the cluster (SigLIP on a CPU-only machine)
3	transcribe speech — context for the gate only, never drawn	Whisper
4	detect ambient sounds with start/end times	BEATs, 527 AudioSet labels, bar 0.35
5	reasoning : is the source visible? what kind of sound? what should the picture show? which sounds are one source?	Qwen2.5-VL-7B
6	draw the picture, compose the side panel	FLUX.1-schnell
7	evaluate	see §6

Behaviours (all decided by models at run time; `config.py` holds every knob, commented)

Visibility — the gate itself (`src/stage5_cross_modal_analysis/reason.py`, `_sound_is_visible`). For each detected sound:

1. Its time span is cut into stretches of at most 5 s. For each stretch, 6 frames are taken from 1 s before to 1 s after it (a short sound is not one frame; the cause is often legible from what changed).
2. Three differently phrased questions go to the VLM about those frames:

- *Name it*: "What in these frames is making the sound of X? Answer with a name, or 'nothing'." If it names something, a text-only follow-up checks world knowledge: "Does a make an X sound?" — so a loosely related object (a cowboy hat, for a vehicle sound) cannot count as the source.
- *Is it happening*: "Can you SEE X happening — the source in frame and visibly making that sound?" — two options, asked twice with the options swapped, counted only if both agree.
- *Describe*: "Describe what is in these frames." If the description names the source, or the world-knowledge check says what it describes could make the sound, that is a vote.

3. Majority of the three votes = visible for that stretch. The sound is silenced only if **every** stretch is visible; if the source enters the frame midway, the picture stays for the whole sound (one continuous sound → one continuous picture, no flicker).

4. A sound that is a *kind* of a silenced sound (Giggle under Laughter, via the ontology) is silenced too.

"Visible" means *seen making the sound*: a baby held in its mother's arms is not visibly crying; a fire-alarm box on a wall is not visibly alarming; a woman with her head back is visibly laughing. Stage 2's object list never silences on its own when the VLM is available — it only flags candidates for the VLM to confirm.

Timing. A sound's start is stamped when its detector score begins to rise, not when it crosses the bar ("hysteresis": an approaching helicopter scores 0.15–0.42 for 1.5 s before crossing 0.35). This cut the timing error from 0.43 to 0.31 s on DCASE gold. A second adjustment, "occlusion" (silence the opening of the window step by step and see when the score drops; `ONSET_CAM`), was measured to add nothing. A picture is held at least 1.5 s; bursts of the same sound closer than 0.8 s are one appearance; the panel is empty when nothing is heard; at most three pictures at once.

What the picture shows (`decide_subjects` in `reason.py`, then stage 6). After the gate says "show":

1. The VLM writes a 2–5-word phrase of the *event*, not the object — "glass shattering", "audience clapping" — from three inputs: the detector's label ("Glass"), the frames spanning the sound (to pick the *kind*: a window or a bottle), and a *place* of at most four words asked for separately ("kitchen"). The transcript is never an input (it leaked dialogue into pictures: "hiccup on the

phone, slow down"), and no example phrases are given (they biased every answer toward the examples).

2. A forced-choice check: "Given this phrase and the other sounds detected in this clip, plus 'none of them', which sound does it depict?" — asked twice with the orders swapped. If the phrase does not point back to its own sound, it is rewritten once.
3. FLUX.1-schnell draws the phrase at 768 px on a plain white background; the side panel shows it for the sound's duration.

One picture per source (`_dedup` and `_disambiguate` in `reason.py`, between the gate and the drawing). The detector often emits a family and its members for one sound (Laughter, Giggle, Snicker). Three mechanisms, in order:

1. **The AudioSet ontology** (vendored as `src/audioset_parents.json`): if one detected label is an ancestor of the other (Laughter → Giggle; Owl → Hoot), they are one source. A family label explains a member only where they overlap in time, so an all-clip "Bird" does not swallow a five-second "Owl". No threshold; 14/14 on every pair seen.
2. **Text similarity of the two phrases** (SigLIP, ≥ 0.80 , only for sounds overlapping in time) for paraphrases the ontology cannot see; known to be coarse (catalogue #14).
3. **Disambiguation**: two labels with the same start and end (a crying baby heard as "Sheep" and "Baby cry") are one sound with two names — the VLM looks at the frames and picks one, or keeps both if it cannot tell.

Speech as context. If speech occurs within 3 s of a sound, a text-only question asks whether the people are reacting to it ("whoa, look at that"). A yes puts the sound first when more sounds overlap than the panel can hold, and can rescue a sound just under the bar. It never reaches the picture and never overrides visibility.

4. How it was built, and what was learned

The system was developed on demo clips, and almost every early fix was wrong on the next clip. The reusable lessons, each with the incident that taught it:

1. **Fixed vocabularies fail.** Visibility was first decided by a hand-written list of ~30 sound labels, each mapped to a CLIP prompt ("siren" → "an ambulance"). Any sound not in the list — laughter, footsteps, a telephone, an owl — could *never* be marked visible, so it got a picture even with its source in plain view.

The same happened with a table of "setting phrases" and a 134-entry synonym table: the entries that matter are the ones nobody thought to write. Every table was replaced by a run-time question to a model about the sound that was actually detected.

2. **Ask for a name, not a yes.** A yes/no question to a VLM collects agreement, not evidence — an early check approved "man on motorcycle drives past silver van" as a picture of *laughter*, because as a sentence about the scene it is fine. Asking "what is making the sound?" produces something checkable (does a van make laughter?). Separately, the 7B model has a position bias — in two-option questions it picked option (b) 22 times in 23 whatever the content — so every two-way question is asked in both orderings and counts only when both agree.
3. **Read structured answers instead of inferring them.** Three ways of deciding "are these two labels one sound?" failed: label embeddings (Dog/Cat 0.90 above Laughter/Snicker 0.80), asking the VLM (it answered "different" 90% of the time — literally correct, useless), and a similarity threshold (re-tuned three times, invalidated each time a prompt elsewhere changed). The AudioSet ontology, shipped with the detector all along, solved it with no threshold.
4. **Set thresholds once on a dev split.** Until 2026-09-13 every knob was retuned on the clip in front of us and broke on the next one. Now (`docs/plan_robustness.md`): thresholds are set once on the 174 dev clips; every failure seen in a demo is logged in `docs/failure_catalogue.md` with clip, stage and confidence; a design change needs three cases of one pattern or a dev-split metric; demos illustrate and never tune.
5. **Evaluation code produces plausible wrong numbers.** An empty augmentation scored a charitable 2/4; a text-similarity helper returned exactly 0.0 for everything after a library upgrade (so deduplication silently stopped); an audio model received no audio because a keyword had been renamed and answered "none" for 100 clips; a job step died on a Windows line ending after a mid-run file sync. Each was found by reading logs, each is now guarded (abstention scored in code, helpers return None instead of 0, assertions on processor output, CRLF stripped on upload). A single number from an automatic pipeline deserves less trust than its precision suggests.
6. **The rubric shapes the system.** A withheld picture scores 0 where one would score up to 4; a redundant picture scores about 3 where silence scores 4. A gate must be right ~80% of the time when silent just to break even, which is

why "the gate wins where nothing is missing and loses where something is" is the shape of every result below.

5. The benchmark (`benchmark/tags.json`, clips in `data/input/benchmark/`)

The clips. 274 short videos (10–30 s), hand-labelled by Adam with a tagging tool (`benchmark/tagger.py`) into four scenarios:

- **unseen ambient** — an ambient sound whose source is off screen: a picture is due;
- **mixed** — some sources visible, some not: a picture is due for the unseen ones;
- **seen ambient** — every ambient sound's source is on screen: no picture due;
- **no ambient** — speech or music only: no picture due.

The labelling rule is the one professional captioners use (DCMP): not "is the physical source in frame?" but "does the picture already tell the viewer this sound is happening?" — wind roar over a rider on a moving motorcycle is *seen*; waves are seen when the sea fills the frame. Labels live in `benchmark/tags.json`; clips in `data/input/benchmark/<scenario>/`.

Splits. The **test set** is 100 clips, 25 per scenario, sampled with a fixed seed (this is what every reported number uses). The **dev split** is the other 174 labelled clips, used for setting thresholds. All 25 mixed clips are in the test set and none in dev — a limitation that later stopped one experiment (§7).

Where they came from, and why 274 not 300. Four sourcing strategies were tried (keyword scraping of Wikimedia/YouTube, AudioSet clips with strong labels, curated film scenes, UnAV-100 overlap moments) plus a three-stage automatic filter; usable positives were 15–20% of candidates under every strategy, and the rate fell as filtering grew. Reported as a finding: off-screen ambient sound is common in video (a third of clips with any ambient sound) but the acquisition funnel discards it. The first 209 tags were made before any model-based pre-screening and are the unbiased sample; later candidates were pre-screened (`benchmark/screen.py`).

Label stability. Sixty tagged clips were re-labelled blind: self-agreement 78%, $\kappa = 0.60$ ("moderate"). The instability sits in the positive class — unseen reproduced 12/20, seen 19/20 — which is exactly the class the gate is judged on. One annotator; a second would bound this.

External gold: DCASE 2025 Task 3. A public research dataset (Sony / Tampere / QMUL): 30,000 five-second clips recorded in real rooms with a 360° camera and a microphone array, cut into normal-looking videos. Every sound event (13 classes — door, knock, footsteps, telephone, clapping, laughter, water tap, bell, music, speech...) is labelled by humans with its start and end at 100 ms resolution, its direction, and whether its source lies **inside the camera's field of view** (on-screen) or not. We did not make these labels, so they are an outside check on two of our components: the visibility question (258 events: agrees 66.7%; keeps 97% of off-screen sounds, silences only 35% of on-screen ones — timid in the safe direction) and the onset timing (255 events, §7). Caveat: their "on-screen" is geometric (direction inside the frame), not "you can see it making the sound", so part of the 35% is the check answering a stricter question than the label. 222 audio files were fetched to `data/dcaset2025_task3/` (the videos carry no audio track; audio is fetched by range request from the 10 GB zip, `benchmark/eval_dcaset_onset.py --fetch-only`).

6. How it is evaluated (this decides what the numbers mean)

There is no human study; the evaluation is fully automatic (proposal §6.1), run by `benchmark/run_protocol.py` in four passes (render → describe → judge → grounded judge). Per clip and system:

1. The system produces its output: the augmented video (pictures beside the frames), or nothing.
 2. A VLM (Qwen2.5-VL) looks at the augmented frames and writes what a viewer would learn from the pictures alone — the **description** ("a viewer would learn that a siren is sounding nearby").
 3. A **reference** sentence states what a hearing viewer gets from the soundtrack that a deaf viewer would miss from the picture alone — or "nothing beyond the picture" when nothing is missing.
 4. An independent judge (Mistral-7B, a different model family from the describer) scores how well the description conveys the reference, 0–4. Two cases are decided in code, not by the judge: showing nothing when something was missing = 0; showing nothing when nothing was missing = 4 (a pilot judge had given empty output a charitable 2).
- **Baselines:** **blind** (draw every sound, never look) and **caption** (text instead of a picture).
 - **Judge dependence:** a second judge reproduces the gated-vs-blind gap ($\kappa =$

0.753) but moves the caption baseline by +0.61 — read gated-vs-blind as robust, gated-vs-caption as a property of the judge.

- **Stats:** `python scripts/paired_stats.py v3 v3_grounded` prints everything, paired with bootstrap CIs, pooled and split by "picture needed" (unseen+mixed) vs "not needed" (seen+no-ambient).

The reference is the crux. Who writes the sentence that says what is missing decides what the numbers mean. Three versions exist; none is neutral, and that is itself a result:

- **model-derived** — what the proposal specified: the system's own VLM writes the reference from the detector's own events. Found circular in review (2026-09-14): it rewards whichever system repeats the detector, i.e. the blind baseline, by about half a point.
- **human-grounded** — the same reference corrected by Adam's label: on *seen* and *no ambient* clips it becomes "nothing beyond the picture". This is the only reference that can score the gate decision, but it needs labels, so a deployed system could never use it. All headline numbers use it, with the model-derived one shown alongside.
- **independent** — built overnight 2026-09-14 from models that never see any system's output: Qwen2-Audio lists the sounds (on the mix and on a Demucs no-vocals stem), CLAP verifies each claim against the audio, Idefics3 (a different VLM) says what is visible, and Phi-3.5-mini writes the sentence (Llama-3.1 was licence-gated at the time; access granted since, not re-run). It removes the circularity but its own "is the source visible?" decision agrees with the human tag on only 55/100 clips — chance — under two variants tried, so it rewards showing. Reported (`v2_indep_list`); not used for v3.

7. Results (v3 = the shipped system; 100 test clips; 0–4)

reference	gated	blind	caption	gated – blind (95% CI)
model-derived	2.64	3.04	2.79	−0.40 [−0.66, −0.16]
human-grounded	3.17	3.16	2.89	+0.01 [−0.15, +0.18] — tie (21 wins / 64 ties / 15 losses)

By scenario (grounded reference):

- **no picture due:** gated 3.56 vs blind 3.30 → +0.26 [+0.12, +0.40] — the gate wins.
- **picture due:** gated 2.78 vs blind 3.02 → −0.24 [−0.54, +0.04] — the gate

loses.

- Gated beats caption by +0.28. A perfect gate would score 3.25.

In words: *the system does the right thing when nothing is missing, and still sometimes stays silent when it should not*. "Gate wins" means: on clips where the source is on screen, the gated system correctly showed nothing (scored 4) while the blind baseline drew a redundant picture (scored ~3). "Gate loses" means: on clips where a picture was due, the gate sometimes showed nothing (scored 0) while blind drew it (scored ~3) — a loss of 3–4 points against a gain of 1, which is why the two strata nearly cancel.

v2 → v3. The earlier configuration v2 (2026-09-12: PANNs detector, a 30-item visibility list, object depictions) lost clearly under every reference (−0.24 to −0.70), and on v2 even a perfect gate would have beaten blind by at most +0.05. The week's changes — BEATs, the per-sound three-vote visibility, event-framed depictions, ontology deduplication, speech as context — closed the gap to a tie. Both runs are in `benchmark/protocol_results_*.json`.

Where the remaining errors come from (read from cached decisions, no GPU): of 11 wrongly-silenced test clips, 7 are the **detector** (ambient sound under speech/music never reached the bar), 4 are label ambiguity (cars visible vs off-screen traffic), 0 are VLM timing. On dev, ~80 of 100 redundant pictures are confident detector **phantoms** (whale at a Christmas market) the VLM rightly cannot silence. So the detector, not the visibility judgment, bounds the result.

What was tried against that, and why it stopped (all on 2026-09-14, each decided with an independent reviewer):

- **Gate threshold sweep** (`benchmark/gate_dev_sweep.py`): the three visibility votes of every sound were cached for all dev and test clips, and 16 settings (majority vs unanimous vote × display bar 0.25–0.40 × kinds rule on/off) were scored on dev with the rubric's own costs (missed picture −4, redundant −1). The shipped setting was already the optimum, and on dev the gate is not timid at all (1 miss in 23 unseen clips) — the sweep could not see the test failure because dev has no mixed clips. No v4 (`benchmark/gate_setting.json`).
- **CLAP second opinion** (`src/stage4_audio_event_detection/clap_check.py`): keep a detection only if a second audio model (CLAP) ranks its sound family near the top for that audio. The cut-off was to be set on DCASE gold at 95% recall and had to be ≤ 30 of 328 families, both declared before running. CLAP reached 50% recall at 30 (footsteps median rank 261) → the check cannot filter phantoms without dropping real sounds. Reported

as a measured limitation of zero-shot audio-text models (`benchmark/clap_setting.json`).

- **Demucs separation** before detection: assessed by two reviewers arguing opposite sides and not run — Demucs removes speech only (4 of the 7 masked misses are under music) and taking the higher of two scores can only add phantoms. First future-work item, with its acceptance bars written in the report.

Other measured results: onset timing on DCASE gold — of 255 gold onsets only 66 are matched by a detection at the 0.35 bar (detector recall on 5 s clips is low); on those, hysteresis cut the error from 0.43 s to 0.31 s and the occlusion refinement adds nothing (`benchmark/eval_dcase_onset.json`); generation vs stock-photo retrieval (+0.17 for retrieval, but 55% restrictive licences).

8. What is done, what is open

Done: pipeline, benchmark, evaluation protocol with three references, all runs (v2, v3), onset and visibility evaluations on DCASE gold, the diagnosis, the report (14 pages, no placeholders), LIMITATIONS.md, this summary. All on `main`, pushed, including the cached gate votes and description caches the diagnosis depends on.

Open decisions for Adam:

- **~~ONSET_CAM~~** — decided 2026-09-15: kept on (no measurable timing difference; v3 was evaluated with it on; it leaves slightly fewer onsets more than half a second off).
- **Independent reference for v3**: run it for table completeness (~1.5 GPU-hours) or skip, given it cannot score the gate.
- **Report**: read-through and submission; the independent reference's Llama writer could be re-run now that access is granted (cosmetic).

Future work, in order (also in the report): a detector-only benchmark on the labelled clips (recall of off-screen events vs phantom rate) and against it a speech-aware detector (Demucs residual, with the declared bars) or a polyphonic detector; an open-vocabulary audio model where AudioSet has no word ("phone alert"); a second annotator; a study with deaf viewers.

9. Where everything is, and how to run it

- Run one video: `python main.py <video>` (see `README.md`); all knobs in

`config.py`.

- **Results:** `benchmark/protocol_results_<tag>.json` (v2, v2_grounded, v2_indep_list, v3, v3_grounded), `benchmark/paired_stats.json`; cached gate votes `benchmark/gate_votes/`.
 - **Cluster** (BIU Slurm; VPN is F5 at access.biu.ac.il, resource AC-Users; `slurm/RUNBOOK.md`): `bash slurm/sync_data.sh --code-only` uploads code (and strips Windows line endings, which otherwise kill job steps). You need your own Hugging Face token exported in `~/.bashrc` on the cluster (FLUX, Llama-3.1 are gated — accept their licences on HF first).
 - **The full experiment:** `sbatch slurm/job_v3.sh` → four stamped passes of `slurm/job_protocol.sh` (render / describe / judge / grounded judge; ~10 h on one L4; resumable from stamps — resubmit, never restart) → `benchmark/protocol_results_v3*.json` → `python scripts/paired_stats.py v3 v3_grounded`.
 - **Documents:** `docs/report/` (thesis draft), `docs/project_notes.tex` (everything, dated), `docs/night_report_2026-09-14.md` (the overnight run), `docs/failure_catalogue.md`, `docs/plan_robustness.md` (process rules), `LIMITATIONS.md`.
-